

SLIM SERIES · 01

심포니에 들어서며

라라벨을 지나온 PHP 개발자를 위한 작은 안내서
Symfony 8.0 · PHP 8.5

차례

머리말	3
-----------	---

1부 — 두 풍경

1. 라라벨이 길러준 직관	5
2. 심포니라는 풍경	6
3. PHP 8.5라는 시대	7

2부 — 독서 일지를 짓다

4. 시작	9
5. 디렉터리를 읽다	10
6. 첫 라우트, 첫 응답	11
7. 컨트롤러는 함수다	12
8. 요청의 인생	13
9. Twig — 보여주는 일은 따로	14
10. Book이라는 사물	15
11. Doctrine과 마이그레이션	16
12. 데이터의 모양	17
13. 목록을 짓다	18
14. 폼 — 입력의 고요함	19
15. 폼의 인생	20
16. 검증과 작은 친절	21
17. 수정과 삭제	22
18. 책의 인생	23
19. AssetMapper	24
20. 콘솔 — 작은 자동화	25
21. 띄워 보다	26

3부 — 다음 한 걸음

22. 가져갈 직관, 두고 갈 습관	28
23. 더 깊은 방들	29
닫는 글 · 콜로폰	30

머리말

이 책은 라라벨을 충분히 만져본 PHP 개발자를 위해 쓰였다. 우리는 이미 컨트롤러를 짓고, 마이그레이션을 만들고, Blade로 화면을 그릴 줄 안다. 그 손이 심포니라는 다른 도시에 닿았을 때 우왕좌왕하지 않도록, 길의 폭과 길의 이름을 적어 두는 일이 이 책의 일이다.

라라벨은 친절하 동네다. 길이 잘 닦여 있고, 어디에 무엇이 있는지 보이지 않는 손이 미리 정리해 둔다. 그 친절함이 우리를 빠르게 자라게 했다.

심포니는 그보다 조금 더 조용한 동네다. 길이 적게 닦여 있다는 뜻이 아니라, 우리가 길을 직접 고를 수 있다는 뜻이다. 라라벨이 거실이라면, 심포니는 가구 없이 비워 둔 큰 방에 가깝다. 어디든 책상을 둘 수 있다.

서른 쪽이라는 분량은 의도된 제약이다. 심포니의 모든 방을 안내하기에는 턱없이 부족하다. 그래서 이 책은 한 가지 작은 일을 함께 한다. 우리가 읽은 책을 기록하는 작은 웹 애플리케이션을 짓는 것이다. 이름은 '독서 일지'.

일을 마치고 나면 두 가지가 남는다. 하나는 작동하는 코드 한 폴더, 다른 하나는 심포니의 골격에 대한 감각이다. 두 번째가 사실은 더 오래 남는다.

코드는 읽을 만큼만 쓴다. 명령어는 따라칠 만큼만 적는다. 다이어그램은 머릿속에 남을 만한 것만 그린다. 나머지 시간은 읽고 생각하는 데 쓰자.

1부

두 풍경

1. 라라벨이 길러준 직관

라라벨에서 우리는 이미 많은 것을 배웠다. `Route::get`으로 길을 깔고, 컨트롤러에서 Eloquent로 데이터를 꺼내고, Blade로 화면을 그렸다. `php artisan make:` 한 줄이면 새 파일이 자리 잡았고, `.env` 한 줄을 바꾸면 모든 환경이 따라 움직였다.

이 직관 대부분은 심포니에서도 그대로 통한다. 다만 이름이 다르고, 위치가 다르다.

라라벨에서 `Route::get`은 심포니에서 `#[Route]`라는 PHP 속성이다. Eloquent는 Doctrine이라는 ORM으로 바뀐다. Blade 자리엔 Twig이 앉는다. `artisan`은 `bin/console`이다. 서비스 컨테이너라는 말을 라라벨에서 들어본 적이 있다면, 심포니에서는 그것이 첫째 시민이다.

다른 것은 두 가지다.

첫째, 심포니는 마법을 덜 쓴다. Eloquent의 `User::find(1)`처럼 정적 메서드로 데이터에 접근하지 않는다. 대신 `BookRepository`라는 클래스를 받아서 `$repo->find(1)`이라고 쓴다. 명시적이라서 처음엔 답답하고, 나중에는 안전하다.

둘째, 심포니는 결정을 미룬다. 라라벨이 좋은 기본값을 정해 두는 것이라면, 심포니는 우리에게 결정을 묻는다. 그 결정이 점점 쌓여서 나만의 도시가 된다.

이 작은 책은 그 도시의 거리 하나를 함께 걸어 보는 일이다.

2. 심포니라는 풍경

심포니의 한가운데에는 다섯 개의 사물이 있다.

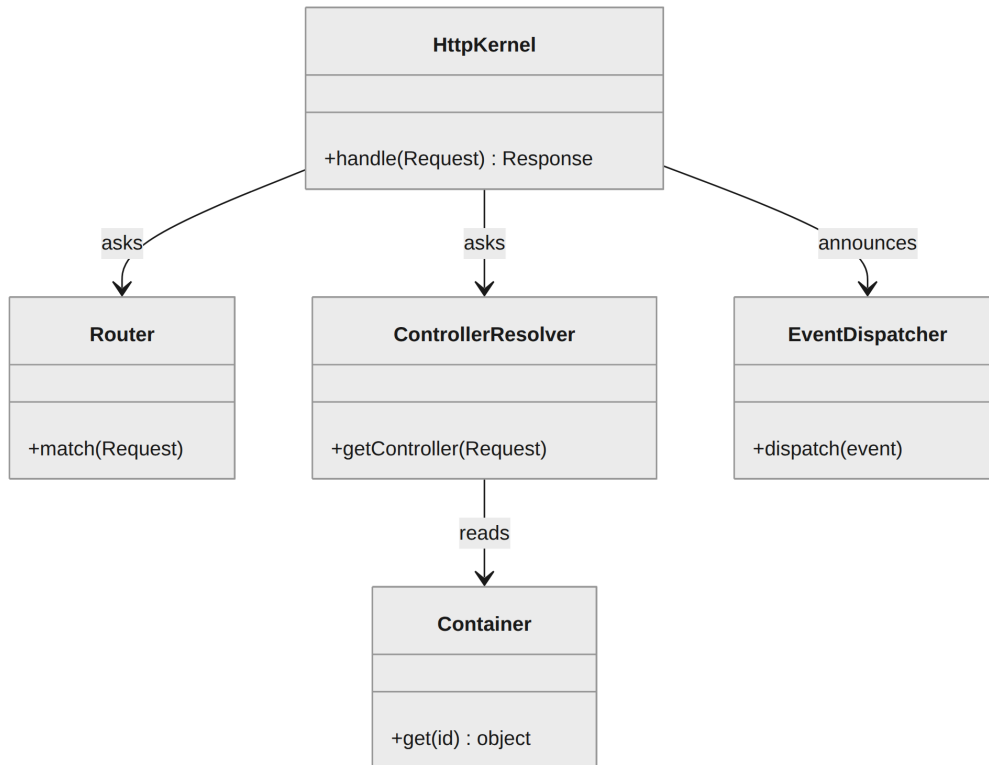


그림 1. 한 요청을 둘러싼 다섯 사물

HttpKernel은 모든 요청을 받는 손이다. 우리는 컨트롤러를 찾고 라우트를 정하지만, 그 모든 것이 모이는 곳은 결국 이 클래스의 `handle()` 메서드다.

Router는 URL을 컨트롤러로 옮기는 사전이다. **ControllerResolver**는 그 컨트롤러 객체를 실제로 꺼내 오는 일을 한다. 객체를 꺼내려면 의존성도 함께 꺼내야 하는데, 그 일은 **Container**가 한다. 처음부터 끝까지, 어떤 단계에 도달할 때마다 **EventDispatcher**가 모든 구독자에게 그 사실을 알린다.

이름이 많지만, 이름들이 하는 일은 단순하다. 받고, 옮기고, 꺼내고, 실행하고, 알린다. 라라벨에도 같은 일이 있었다. 이름만 달랐을 뿐이다.

3. PHP 8.5라는 시대

이 책은 PHP 8.5.5를 기준으로 쓴다. 2026년 3월에 나온 안정 버전이다. 8.5는 작지만 즐거운 변화를 가져왔다. 두 가지를 짚어 두자.

파이프 연산자

```
$slug = $title
    |> strtolower(...)
    |> trim(...)
    |> fn ($s) => str_replace(' ', '-', $s);
```

위에서 아래로 읽힌다. 라라벨의 `Str::of()->lower()->trim()->slug()` 체인이 코어 문법으로 들어왔다고 보면 된다.

clone with

```
$updated = clone $book with {
    status: 'finished',
    readAt: new DateTimeImmutable(),
};
```

원본은 그대로 두고 새 객체만 바꾼다. readonly 클래스와 함께 쓸 때 가장 빛난다. 함수형 프로그래밍의 깔끔함을 객체지향 안에 들여놓은 셈이다.

이 책에서 두 기능을 적극적으로 등장시키지는 않는다. 다만 이렇게만 알아 두자. 우리가 쓰는 도구가 점점 깔끔해지고 있다는 사실을.

2부

독서 일지를 짓다

4. 시작

심포니는 자신의 CLI를 가지고 있다. macOS, Linux 모두 동일하다.

```
curl -sS https://get.symfony.com/cli/installer | bash
```

설치가 끝났다면 환경을 점검한다. PHP 8.4 이상이 필요하다.

```
symfony check:requirements
```

문제가 없다면 새 프로젝트를 만든다. 우리는 웹 애플리케이션을 만들 것이므로 `--webapp` 옵션을 더한다.

```
symfony new reading-log --version="8.0.*" --webapp
cd reading-log
symfony serve -d
```

`-d`는 데몬으로 띄운다는 뜻이다. 브라우저로 `https://127.0.0.1:8000`을 열면 심포니의 환영 페이지가 보인다. 라라벨의 `php artisan serve`와 거의 같은 첫 인상이다.

여기서 잠깐. 화면이 비어 있다고 당황할 필요는 없다. 우리는 그 화면을 곧 우리의 손으로 그릴 것이다.

5. 디렉터리를 읽다

새 프로젝트를 열어 보면 폴더 구조는 이렇다.

```
reading-log/
├─ bin/console      # artisan에 해당
├─ config/          # 라우트, 서비스 등의 설정
├─ public/          # 웹 루트, index.php
├─ src/             # 우리가 쓸 PHP 코드
├─ templates/       # Twig 템플릿
├─ migrations/      # Doctrine 마이그레이션
├─ var/             # 캐시, 로그
├─ vendor/          # composer 의존성
└─ .env             # 환경 변수
```

라라벨과 비슷하지만 미묘하게 다르다. `app/` 대신 `src/` 이고, `routes/` 폴더는 없다. 라우트는 컨트롤러의 PHP 속성으로 직접 박아 두기 때문이다.

`config/` 안의 파일들을 한 번 열어 보길 권한다. PHP 배열로 작성된 설정이 보일 것이다. 심포니 8부터는 YAML이 기본 자리에서 물러났다. 우리가 쓰는 PHP가 설정도 같이 한다.

6. 첫 라우트, 첫 응답

가장 작은 컨트롤러를 만들어 보자.

```
// src/Controller/HelloController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

class HelloController extends AbstractController
{
    #[Route('/hello')]
    public function __invoke(): Response
    {
        return new Response('안녕, 심포니.');
```

브라우저에서 `/hello`를 열면 그 한 줄이 보인다. 라라벨로 치면 다음과 같은 코드다.

```
Route::get('/hello', fn () => '안녕, 라라벨.');
```

다른 점이 한눈에 들어온다. 라라벨은 길과 응답을 한 곳에 묶었고, 심포니는 그 둘을 클래스의 PHP 속성으로 묶었다. 어느 쪽이 옳다고 말하기는 어렵다. 다만 심포니의 방식은 컨트롤러가 자기 길을 직접 안다는 뜻이다.

7. 컨트롤러는 함수다

심포니 8에서 가장 권장하는 컨트롤러 모양은 메서드 하나짜리 클래스다.

```
class ShowBookController extends AbstractController
{
    #[Route('/books/{id}', name: 'book_show')]
    public function __invoke(
        int $id,
        BookRepository $books,
    ): Response {
        $book = $books->find($id);
        if (!$book) {
            throw $this->createNotFoundException();
        }
        return $this->render('book/show.html.twig', [
            'book' => $book,
        ]);
    }
}
```

`__invoke`라는 이름이 낯설 수 있다. 객체를 함수처럼 호출할 수 있게 만드는 마법 메서드다. 즉, 이 컨트롤러 클래스 자체가 곧 하나의 함수다. 컨트롤러 하나는 보통 한 가지 일만 한다. 책을 보여주는 일, 책을 만드는 일, 책을 지우는 일은 서로 다른 클래스다. 한 클래스가 한 가지 일을 한다는 약속은, 그 클래스를 다시 읽을 때의 친절함으로 돌아온다.

8. 요청의 일생

한 요청이 들어와 응답이 되어 나가는 길은 짧지 않다.

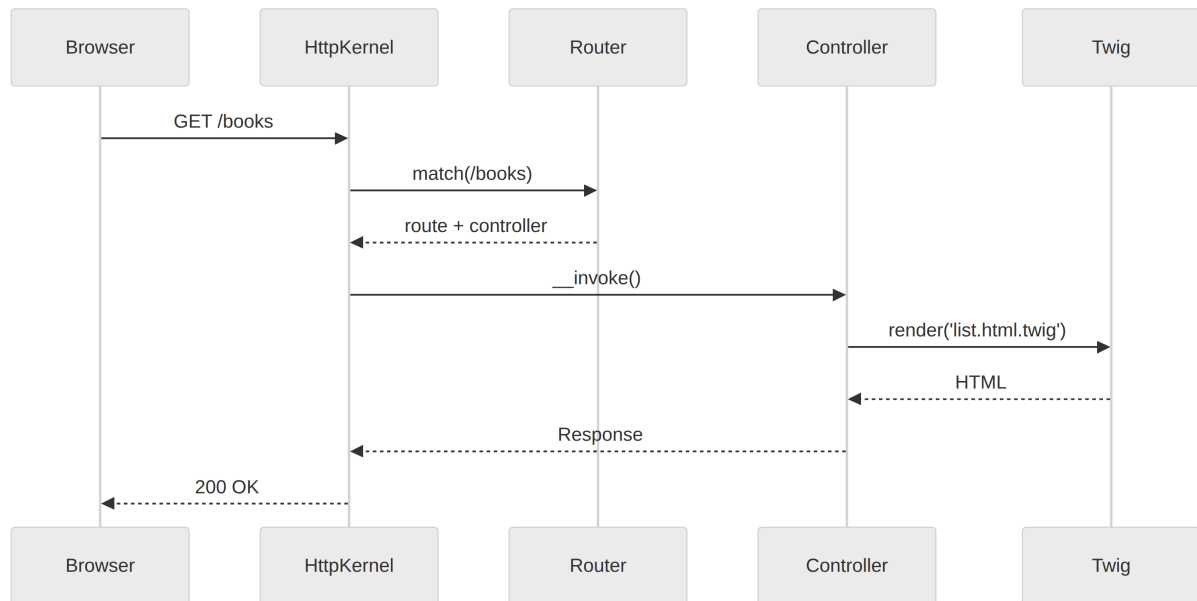


그림 2. 요청 하나가 거치는 다섯 손

HttpKernel이 받고, Router에게 길을 묻고, 그 길을 따라 컨트롤러를 깨운다. 컨트롤러는 자기 일을 마치고 Twig에게 화면을 부탁한 뒤, Response를 손에 쥐고 다시 HttpKernel에게 돌아온다. 그제서야 응답이 브라우저로 떠난다.

읽고 나면 단순하다. 다만 이 길의 어느 지점에서든 `EventDispatcher`가 사람들을 깨운다는 점만 기억하자. 우리는 그 사람들 중 하나가 되어, 우리만의 일을 슬쩍 끼워 넣을 수 있다. 그것이 심포니다움이다.

9. Twig — 보여주는 일은 따로

심포니에서는 `templates/` 폴더에 Twig 파일을 둔다.

```
{# templates/base.html.twig #}
<!DOCTYPE html>
<html lang="ko">
<head>
  <title>{% block title %}독서 일지{% endblock %}</title>
</head>
<body>{% block body %}{% endblock %}</body>
</html>
```

자식 템플릿은 `extends`로 이 뼈대를 받는다.

```
{# templates/book/list.html.twig #}
{% extends 'base.html.twig' %}
{% block body %}
  <h1>나의 독서 일지</h1>
  <ul>
    {% for book in books %}
      <li>{{ book.title }} - {{ book.author }}</li>
    {% endfor %}
  </ul>
{% endblock %}
```

Blade와 비교하면 `@`가 `{% %}`로, `{{ }}`는 그대로다. 자동 이스케이프도 같다. 다만 Twig은 PHP를 직접 쓰지 못한다. 메서드 호출은 `book.title`처럼 점 하나로 충분하다.

10. Book이라는 사물

우리가 다루려는 가장 작은 사물은 '책'이다.

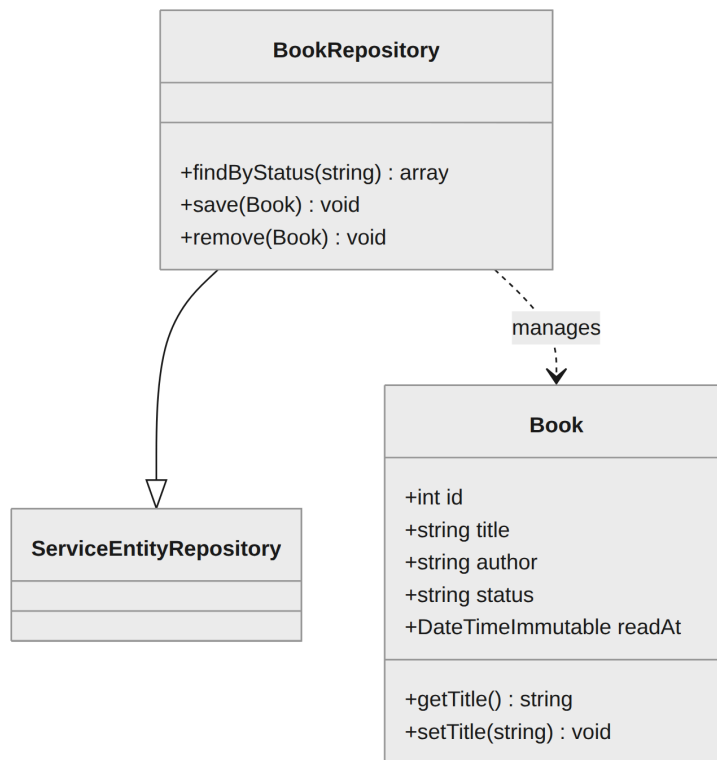


그림 3. 사물과 그 사물을 다루는 손

Book 클래스는 데이터의 모양만 담는다. 제목, 저자, 상태, 그리고 다 읽은 날짜. 그 외의 일은 클래스가 하지 않는다.

데이터를 꺼내고 저장하는 일은 **BookRepository**가 맡는다. 라라벨의 Eloquent에서 `Book::where(...)->get()`이 한 클래스 안에서 이루어졌다면, Doctrine은 그 일을 두 클래스로 나누어 둔다. 모델은 데이터, 리포지토리는 행위. 이 분리가 처음에는 번거롭지만, 도메인이 자라날수록 고마워진다. **Book** 클래스는 ORM을 모르는 채로 살아갈 수 있고, 우리는 그 사실을 즐길 수 있다.

11. Doctrine과 마이그레이션

엔티티를 PHP로 적는 모양은 다음과 같다.

```
// src/Entity/Book.php
namespace App\Entity;

use App\Repository\BookRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: BookRepository::class)]
class Book
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 200)]
    private string $title = '';

    #[ORM\Column(length: 120)]
    private string $author = '';

    #[ORM\Column(length: 20)]
    private string $status = 'want_to_read';
}
```

이 파일을 적었다면, 다음 두 줄을 차례로 부른다.

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

심포니는 우리가 적은 엔티티와 현재 스키마를 비교하여, 차이만큼만 자동으로 마이그레이션을 그려 준다. 우리가 직접 컬럼을 적지 않아도 된다.

12. 데이터의 모양

독서 일지의 데이터는 두 테이블이다. 사용자 한 명이 여러 권의 책을 가진다.

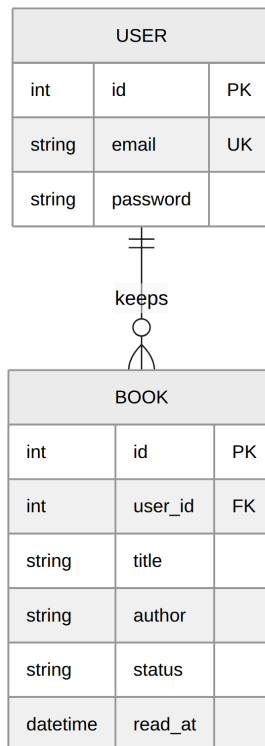


그림 4. 한 사람의 책장을 두 테이블로

이 그림은 우리가 두 번째 학기에 이미 그려본 것이다. 데이터베이스 강의의 첫 그림. 심포니에서도 같다. 사물은 변하지 않았고, 사용하는 도구만 바뀌었다.

USER 테이블은 심포니가 보안 모듈을 위해 자동으로 만들 수도 있고, 우리가 직접 적을 수도 있다. 30쪽이라는 약속이 있으므로, 이 책에서는 한 사용자만 있다고 가정한다. 그 한 사람이 자기의 책장을 채운다.

13. 목록을 짓다

책을 모두 보여주는 페이지를 만들자.

```
// src/Controller/ListBooksController.php
namespace App\Controller;

use App\Repository\BookRepository;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

class ListBooksController extends AbstractController
{
    #[Route('/', name: 'book_list')]
    public function __invoke(BookRepository $books): Response
    {
        return $this->render('book/list.html.twig', [
            'books' => $books->findBy([], ['id' => 'DESC']),
        ]);
    }
}
```

세 줄짜리 클래스다. `BookRepository`는 인자에 타입을 적는 것만으로 자동으로 들어온다. 라라벨의 자동 주입과 같다. 다른 점은, 우리가 그 객체를 어디서 가져왔는지 코드만 봐도 알 수 있다는 것이다.

브라우저로 `/`를 열면 빈 목록이 보인다. 다음 일은, 그 목록에 책 한 권을 더하는 일이다.

14. 폼 — 입력의 고요함

폼은 두 단계로 만든다. 폼의 모양을 정의하는 클래스, 그리고 그것을 사용하는 컨트롤러.

```
// src/Form/BookType.php
namespace App\Form;

use App\Entity\Book;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\Extension\Core\Type\ChoiceType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class BookType extends AbstractType
{
    public function buildForm(FormBuilderInterface $b, array $opts): void
    {
        $b->add('title', TextType::class, ['label' => '제목'])
        ->add('author', TextType::class, ['label' => '저자'])
        ->add('status', ChoiceType::class, [
            'label' => '상태',
            'choices' => [
                '읽고 싶다' => 'want_to_read',
                '읽는 중' => 'reading',
                '다 읽음' => 'finished',
            ],
        ]);
    }

    public function configureOptions(OptionsResolver $r): void
    {
        $r->setDefaults(['data_class' => Book::class]);
    }
}
```

15. 폼의 일생

폼이 어떻게 흘러가는지 한 장으로 보자.

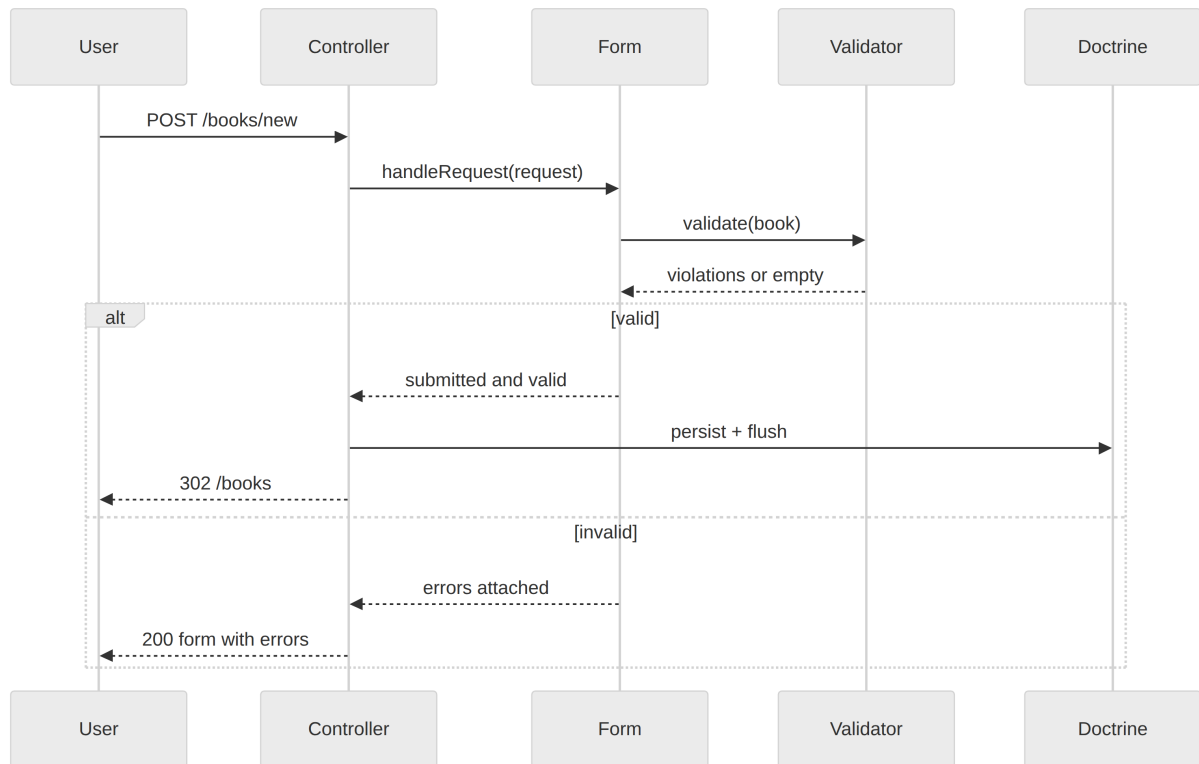


그림 5. 폼이 도는 한 바퀴

사용자가 `POST`로 폼을 보내면, 컨트롤러는 그 요청을 폼 객체에 넘긴다. 폼은 데이터를 채우고, `Validator`에게 검증을 맡긴다. 검증이 통과하면 컨트롤러는 `Doctrine`에게 저장을 부탁하고, 사용자를 목록 페이지로 보낸다. 통과하지 못했다면, 같은 화면에 오류와 함께 폼을 다시 보여준다.

이 모든 일을 라라벨에서는 `FormRequest`와 `validator`가 나누어 했다. 심포니에서는 폼 자체가 두 일을 함께 한다. 그래서 `handleRequest($request)` 한 줄이면, 그 안의 모든 일이 일어난다.

16. 검증과 작은 친절

검증 규칙은 엔티티 자체에 둔다.

```
use Symfony\Component\Validator\Constraints as Assert;

class Book
{
    #[ORM\Column(length: 200)]
    #[Assert\NotBlank(message: '제목이 필요합니다')]
    #[Assert\Length(max: 200)]
    private string $title = '';
}
```

규칙이 데이터의 자리에 있으니, 어느 폼에서 쓰이든 같은 규칙이 따라간다. 한 곳에 적었지만 모든 곳에서 통한다.

검증이 끝난 뒤 사용자에게 작은 친절을 더하자. 한 줄짜리 알림이다.

```
$this->addFlash('success', '책 한 권을 더했습니다.');
```

```
return $this->redirectToRoute('book_list');
```

다음 페이지에서 Twig은 그 알림을 꺼내어 보여줄 수 있다.

```
{% for label, msgs in app.flashes %}
    {% for m in msgs %}<div class="alert">{{ m }}</div>{% endfor %}
{% endfor %}
```

17. 수정과 삭제

수정과 삭제는 새 책을 더하는 일과 닮았다. 다른 한 가지는 어떤 책을 다루는지 분명히 짚어야 한다는 점이다.

```
class EditBookController extends AbstractController
{
  #[Route('/books/{id}/edit', name: 'book_edit')]
  public function __invoke(
    Book $book,
    Request $request,
    EntityManagerInterface $em,
  ): Response {
    $form = $this->createForm(BookType::class, $book);
    $form->handleRequest($request);
    if ($form->isSubmitted() && $form->isValid()) {
      $em->flush();
      $this->addFlash('success', '저장했습니다. ');
      return $this->redirectToRoute('book_list');
    }
    return $this->render('book/edit.html.twig', [
      'form' => $form,
    ]);
  }
}
```

`Book $book` 이라고만 적어 두면 심포니가 알아서 `id` 를 보고 데이터베이스에서 꺼낸다. 라라벨의 라우트 모델 바인딩과 같다. 삭제는 더 단순하다. `$em->remove($book); $em->flush();` 두 줄이면 끝이다.

18. 책의 일생

책 한 권의 상태는 시간에 따라 옮겨 다닌다. 그 옮김을 한 장에 그리면 다음과 같다.

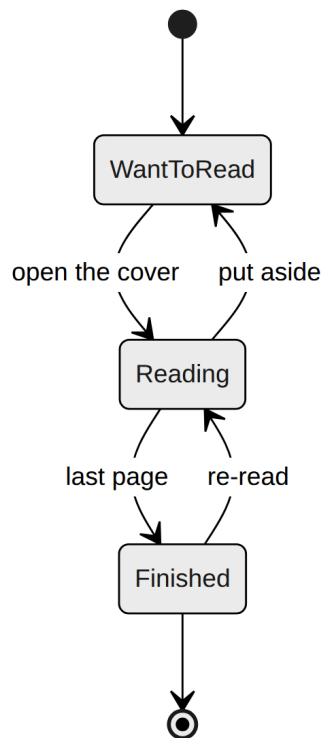


그림 6. 한 권의 책이 거치는 자리들

이 다이어그램은 곧 우리가 위에서 만든 `ChoiceType`의 선택지였다. 그저 데이터의 한 칸이라기보다, 한 권의 책이 우리 곁에서 보내는 시간의 모양이다.

심포니에는 `Workflow`라는 더 정교한 도구가 있다. 상태 전이를 코드로 명시하고, 잘못된 전이를 막는 일을 자동으로 한다. 하지만 우리는 30쪽 안에 있다. 지금은 한 컬럼으로 충분하다. 도구는 필요할 때 부르는 편이 좋다.

19. AssetMapper

라라벨에서 우리는 Vite를 만났다. 자바스크립트와 CSS를 한 데 묶어 빠르게 보내는 도구다. 심포니 8은 비슷한 자리에 `AssetMapper`를 둔다. 다른 점이 있다면, 빌드 단계가 없다는 것이다.

```
php bin/console importmap:require simple-stylesheets
```

이 명령어 한 번이면 `assets/` 폴더와 `importmap.php` 파일이 그 라이브러리를 알게 된다. 우리가 적는 자바스크립트는 그대로 브라우저에 가고, 브라우저는 ESM과 importmap을 이해한다.

```
// assets/app.js
import './styles/app.css';

document.addEventListener('DOMContentLoaded', () => {
    console.log('독서 일지 시작.');
```

Twig에는 한 줄을 더한다.

```
{% block stylesheets %}
    {{ importmap('app') }}
{% endblock %}
```

번들링이 없으니 빌드가 빠르다. 빠른 만큼 변화를 즉시 본다.

20. 콘솔 — 작은 자동화

심포니 8부터는 콘솔 명령어도 invokable 클래스로 적을 수 있다. 보일러플레이트가 한층 줄었다.

```
// src/Command/ImportBooksCommand.php
namespace App\Command;

use Symfony\Component\Console\Attribute\AsCommand;
use Symfony\Component\Console\Style\SymfonyStyle;

#[AsCommand(
    name: 'books:import',
    description: '책 목록을 CSV로 가져온다',
)]
class ImportBooksCommand
{
    public function __invoke(SymfonyStyle $io, string $file): int
    {
        if (!is_readable($file)) {
            $io->error("파일을 열 수 없습니다: $file");
            return 1;
        }
        $io->success("불러왔습니다.");
        return 0;
    }
}
```

실행은 `php bin/console books:import data/books.csv`. 한 줄짜리 import 명령어가 생기면, 다음에는 그것을 매일 새벽에 한 번 부르는 일이 손에 잡힌다. 심포니의 `Scheduler`가 그 일을 돌봐 준다.

21. 띄워 보다

이제 띄울 시간이다.

```
symfony serve
```

콘솔에 보이는 `https://127.0.0.1:8000`을 따라가면 우리가 만든 첫 페이지가 있다. 빈 목록이 보일 것이다. `/books/new`로 가서 책 한 권을 입력하면 목록에 그 책이 자리 잡는다.

데이터베이스는 기본적으로 `var/data.db`라는 파일에 SQLite로 들어간다. 우리가 적은 모든 것이 그 파일 하나에 모인다. PostgreSQL이나 MySQL이 필요하다면 `.env`의 `DATABASE_URL`을 한 줄 바꾸면 된다.

```
DATABASE_URL="postgres://app:secret@db:5432/app"
```

라라벨의 `.env`와 거의 같은 자리에 거의 같은 일을 한다. 심포니의 좋은 점 중 하나는, 우리가 새로 외울 것이 그렇게 많지 않다는 사실이다.

이쯤이면 우리의 독서 일지는 한 사람의 작은 친구가 될 만한 모양을 갖추었다.

3부

다음 한 걸음

22. 가져갈 직관, 두고 갈 습관

라라벨에서 가져갈 만한 직관과, 한 번쯤 의심해 볼 습관을 정리한다.

| 가져갈 것

- 컨트롤러는 얇아야 한다는 감각. 심포니에서도 같다.
- 라우트 모델 바인딩의 편안함. 심포니가 더 적극적으로 챙긴다.
- `.env`에 비밀을 두는 일. 같은 자리에 같은 약속이 있다.
- 마이그레이션을 코드와 함께 커밋하는 습관.

| 두고 갈 것

- 정적 메서드로 모든 일을 끝내려는 버릇. 심포니는 정적이지 않다.
- `make:` 명령어를 외워서 쓰는 습관. 심포니에서는 무엇을 만들지가 더 중요하고, 어떻게 만들지는 도구가 답한다.
- 헬퍼가 자동으로 글로벌에 떠 있을 것이라는 기대. 심포니의 도구는 모두 `import` 한 번을 부른다.

가장 큰 변화는 마음의 자세다. 라라벨이 우리를 위해 결정해 주었다면, 심포니는 우리에게 결정을 묻는다. 첫째철은 답답할 수 있다. 그 답답함이 지나면, 우리는 우리만의 작은 도시를 가진 것이 된다.

23. 더 깊은 방들

이 책 너머로 갈 만한 방들을 짚어 둔다.

Security. 로그인과 권한을 다룬다. 라라벨의 `Auth` 자리에 해당하지만 더 유연하다. `make:user`와 `make:auth`로 시작하면 된다.

Messenger. 큐와 비동기 작업이다. `App\Message\SendEmail` 같은 메시지를 만들고, 그것을 백그라운드에서 처리하게 한다. 라라벨의 `Job`과 같은 개념이다.

Scheduler. 라라벨의 `Kernel::schedule()` 자리. 심포니에서는 `RecurringMessage`로 표현한다.

Mailer와 Notifier. 메일은 `Mailer`, 슬랙이나 SMS는 `Notifier`. 라라벨의 `Mail`과 `Notification`이 한 패밀리로 나뉘어 자란 모습이다.

Workflow. 23쪽에서 잠깐 언급한 그 컴포넌트다. 책의 상태가 더 복잡해진다면, 그때 부르자.

각 방에 들어갈 때마다 한 권의 책이 더 필요할 수 있다. 그러나 그 모든 방의 골격은 우리가 이미 본 것이다. 받고, 옮기고, 꺼내고, 실행하고, 알린다.

답는 글 · 콜로폰

서른 쪽이 짧다고 느꼈다면 그건 잘된 일이다. 책은 짧을수록 다시 펴게 되고, 다시 펴는 책일수록 친구가 된다.

심포니는 우리에게 묻는 도시다. 처음에는 묻는 일이 피곤하다. 점점 자기만의 답이 생기고, 그 답들이 모여서 우리가 만들고 싶은 모양이 된다. 라라벨이 길러 준 직관은, 그 답을 짧게 만든다.

이 책으로 만든 독서 일지가 누군가의 책장을 정리하는 데 한 줄이라도 도움이 되었다면, 그것으로 충분하다. 다음 책은 더 짧을 수도 있다.

— jaywoo, 2026년 5월

콜로폰

본문	Noto Sans KR
코드	Roboto Mono
조판	htmlasitis 0.1.0
다이어그램	mermaid.js (PNG로 사전 렌더)
인쇄	A4 · 30면
저자	jaywoo · 2026